

### Dynamic Programming Problems (Chapter 14)

#### 1. Rod Cutting Problem

##### PROBLEM EXPLANATION

The rod cutting problem is a classic optimization problem where you have a steel rod of length  $n$  and must partition it into smaller pieces by making cuts. Each piece of length  $i$  can be sold for a price  $p[i]$ . The goal is to determine how to cut the rod (or not cut it at all) to maximize the total revenue. This is the foundational example in CLRS Chapter 14 because it demonstrates overlapping subproblems and optimal substructure. You can either sell the rod whole, or make a cut at position  $i$  (receiving price  $p[i]$  for one piece and keeping the remainder of length  $n - i$ ). The key insight is that once you decide to cut at position  $i$ , the remaining problem of maximizing revenue from a rod of length  $n - i$  has the same structure as the original problem.

##### CORE FORMULA

$$r(n) = \max(p(n), \max_{1 \leq i < n} (p(i) + r(n - i)))$$

Or equivalently:  $r(n) = \max_{1 \leq i \leq n} (p(i) + r(n - i))$

Where  $r(n)$  = maximum revenue achievable from a rod of length  $n$ , and  $p(i)$  = price for a rod of length  $i$ . Base case:  $r(0) = 0$ .

##### BOTTOM-UP COMPUTATION

```
for j = 1 to n:
    q = -∞
    for i = 1 to j:
        q = max(q, p[i] + r[j-i])
    r[j] = q
```

**Key Operation:** For each length  $j$ , try all possible first cuts at positions 1 to  $j$ , and keep the maximum revenue.

## 2. Matrix Chain Multiplication (CLRS 14.2)

### PROBLEM EXPLANATION

Given a chain of  $n$  matrices  $A_1, A_2, \dots, A_n$  to be multiplied, the goal is to find the most efficient way to parenthesize the multiplication. The number of scalar multiplications depends on the order of operations, not the grouping itself. For example, if  $A_1$  is  $10 \times 30$ ,  $A_2$  is  $30 \times 5$ , and  $A_3$  is  $5 \times 60$ , then  $((A_1 A_2) A_3)$  requires  $10 \times 30 \times 5 + 10 \times 5 \times 60 = 1500 + 3000 = 4500$  scalar multiplications, while  $(A_1 (A_2 A_3))$  requires  $30 \times 5 \times 60 + 10 \times 30 \times 60 = 9000 + 18000 = 27000$  operations. The problem is to find the parenthesization that minimizes total scalar multiplications. This requires solving overlapping subproblems for all possible subchains and combining them optimally.

### CORE FORMULA

$$m[i, j] = 0 \quad \text{if } i = j$$
$$m[i, j] = \min_{\{i \leq k < j\}} (m[i, k] + m[k+1, j] + p_{i-1} \cdot p_k \cdot p_j) \quad \text{if } i < j$$

Where  $m[i, j]$  = minimum number of scalar multiplications to compute the matrix chain product from  $A_i$  to  $A_j$ , and  $p_{i-1} \times p_i$  = dimensions of matrix  $A_i$ . The split point  $k$  determines where to place the outermost parentheses.

**Optimal Substructure:** The optimal parenthesization of the chain from  $A_i$  to  $A_j$  involves optimally parenthesizing  $A_i$  to  $A_k$ , optimally parenthesizing  $A_{k+1}$  to  $A_j$ , and finally multiplying the resulting two matrices (costing  $p_{i-1} \cdot p_k \cdot p_j$ ).

### BOTTOM-UP COMPUTATION

```
for len = 2 to n:           // len = chain length
  for i = 1 to n-len+1:
    j = i + len - 1
    m[i, j] = ∞
    for k = i to j-1:
      q = m[i, k] + m[k+1, j] + p[i-1] * p[k] * p[j]
      if q < m[i, j]:
        m[i, j] = q
        s[i, j] = k // Store split point for reconstruction
```

**Complexity:** Time:  $O(n^3)$  | Space:  $O(n^2)$

### 3. Longest Common Subsequence (CLRS 14.4)

#### PROBLEM EXPLANATION

Given two sequences  $X = \langle x_1, x_2, \dots, x_m \rangle$  and  $Y = \langle y_1, y_2, \dots, y_n \rangle$ , find the longest subsequence common to both sequences. A subsequence maintains relative order but need not be contiguous. For example, with  $X = \text{"AGGTAB"}$  and  $Y = \text{"GXTXAYB"}$ , the LCS is "GTAB" of length 4. This problem has optimal substructure because if we have  $\text{LCS}(X[1..m], Y[1..n])$ , then either the last characters match (and LCS is  $\text{LCS}(X[1..m-1], Y[1..n-1])$  plus the matching character) or one of them doesn't match (and LCS is the longer of  $\text{LCS}(X[1..m-1], Y[1..n])$  or  $\text{LCS}(X[1..m], Y[1..n-1])$ ). Many overlapping subproblems exist as we explore different prefixes of  $X$  and  $Y$ .

#### CORE FORMULA

$$\begin{aligned} c[i,j] &= 0 && \text{if } i = 0 \text{ or } j = 0 \\ c[i,j] &= c[i-1,j-1] + 1 && \text{if } i,j > 0 \text{ and } x_i = y_j \\ c[i,j] &= \max(c[i-1,j], c[i,j-1]) && \text{if } i,j > 0 \text{ and } x_i \neq y_j \end{aligned}$$

#### BOTTOM-UP COMPUTATION

```
for i = 0 to m: c[i,0] = 0
for j = 0 to n: c[0,j] = 0

for i = 1 to m:
  for j = 1 to n:
    if X[i] == Y[j]:
      c[i,j] = c[i-1,j-1] + 1
      b[i,j] = "\"
    else if c[i-1,j] >= c[i,j-1]:
      c[i,j] = c[i-1,j]
      b[i,j] = "\u2191"
    else:
      c[i,j] = c[i,j-1]
      b[i,j] = "\u2190"
```

#### RECONSTRUCTION

```
PRINT-LCS(b, X, i, j):
  if i == 0 or j == 0: return
  if b[i,j] == "\":
    PRINT-LCS(b, X, i-1, j-1)
    print X[i]
  else if b[i,j] == "\u2191":
    PRINT-LCS(b, X, i-1, j)
  else:
    PRINT-LCS(b, X, i, j-1)
```

**Complexity:** Time:  $O(m \times n)$  | Space:  $O(m \times n)$

## 4. Optimal Binary Search Tree (CLRS 14.5)

### PROBLEM EXPLANATION

Given a set of  $n$  keys  $k_1, k_2, \dots, k_n$  with known search frequencies  $p[1], p[2], \dots, p[n]$  and miss frequencies  $q[0], q[1], \dots, q[n]$ , construct a binary search tree that minimizes the expected search cost. The expected cost is the sum over all keys of the key's depth times its probability, plus the sum of dummy keys' depths times their miss probabilities. The solution involves trying all possible roots: if we choose key  $k_r$  as root, then keys  $k_1$  to  $k_{r-1}$  form the left subtree and  $k_{r+1}$  to  $k_n$  form the right subtree. The recursion accounts for the fact that depths increase by 1 when moving to subtrees.

### CORE FORMULA

$$e[i,j] = q[i-1] \quad \text{if } j = i-1$$
$$e[i,j] = \min_{\{i \leq r \leq j\}} (e[i,r-1] + e[r+1,j] + w[i,j]) \quad \text{if } i \leq j$$

Where  $w[i,j] = \sum_{l=i}^j p[l] + \sum_{l=i-1}^j q[l]$  represents the sum of all probabilities in range  $[i,j]$ . This accounts for all nodes entering the optimal substructure solution getting one level deeper.

### BOTTOM-UP COMPUTATION

```
for i = 1 to n+1:
    e[i,i-1] = q[i-1]
    w[i,i-1] = q[i-1]

for len = 1 to n:
    for i = 1 to n-len+1:
        j = i + len - 1
        e[i,j] = ∞
        w[i,j] = w[i,j-1] + p[j] + q[j]
        for r = i to j:
            t = e[i,r-1] + e[r+1,j] + w[i,j]
            if t < e[i,j]:
                e[i,j] = t
                root[i,j] = r
```

**Complexity:** Time:  $O(n^3)$  | Space:  $O(n^2)$

## 5. Longest Increasing Subsequence (CLRS Exercise 14.3-7)

### PROBLEM EXPLANATION

Given a sequence of integers, find the longest subsequence in which elements are in strictly increasing order (not necessarily contiguous). For example, in [3, 10, 2, 1, 20], the LIS is [3, 10, 20] with length 3. This problem exhibits optimal substructure: if we consider position  $i$ , the LIS ending at position  $i$  can be computed from the LIS values at all earlier positions  $j$  where  $arr[j] < arr[i]$ . We then take the maximum length LIS ending before  $i$  and add 1 for  $arr[i]$ .

### CORE FORMULA

$$lis[i] = 1 + \max(lis[j]) \quad \text{for all } j < i \text{ where } arr[j] < arr[i]$$
$$\text{Final Answer: } LIS\_length = \max_{\{1 \leq i \leq n\}}(lis[i])$$

### BOTTOM-UP COMPUTATION

```
for i = 1 to n:
    lis[i] = 1
    for j = 1 to i-1:
        if arr[j] < arr[i]:
            lis[i] = max(lis[i], lis[j] + 1)
return max(lis[1..n])
```

**Complexity:** Time:  $O(n^2)$  | Space:  $O(n)$  (Can be optimized to  $O(n \log n)$  using binary search).

## 6. Optimal Polygon Triangulation (CLRS 14.5-1)

### PROBLEM EXPLANATION

Given a convex polygon with  $n$  vertices, triangulate it (divide into  $n-2$  triangles by drawing non-crossing diagonals) such that the sum of the weights of all triangles is minimized. This is solved similarly to matrix chain multiplication: for a polygon with vertices  $p_0, p_1, \dots, p_{n-1}$ , if we fix edge  $(p_0, p_{n-1})$  as the base, we choose some vertex  $p_k$  ( $1 \leq k \leq n-2$ ) to form triangle  $(p_0, p_k, p_{n-1})$ , then recursively triangulate the left polygon  $(p_0, p_1, \dots, p_k)$  and right polygon  $(p_k, p_{k+1}, \dots, p_{n-1})$ .

### CORE FORMULA

$$t[i,j] = 0 \quad \text{if } j \leq i+1$$
$$t[i,j] = \min_{\{i < k < j\}} (t[i,k] + t[k,j] + \text{weight}(v_i, v_k, v_j)) \quad \text{if } j > i+1$$

## BOTTOM-UP COMPUTATION

```
for i = 1 to n:
    t[i,i] = 0; t[i,i+1] = 0

for len = 2 to n-1:
    for i = 1 to n-len:
        j = i + len
        t[i,j] = ∞
        for k = i+1 to j-1:
            cost = t[i,k] + t[k,j] + weight(v[i], v[k], v[j])
            if cost < t[i,j]:
                t[i,j] = cost
                split[i,j] = k
```

**Complexity:** Time:  $O(n^3)$  | Space:  $O(n^2)$

## 7. Assembly Line Scheduling

### PROBLEM EXPLANATION

A factory has two assembly lines, each with  $n$  stations. Line  $i$ , station  $j$  has processing time  $a_{i,j}$ . Switching from line 1 to line 2 after station  $j$  costs  $t_{1,j}$ . The goal is to find the fastest path through all stations. At each station, you decide whether to stay on your current line or switch to the other line.

### CORE FORMULA

$$f[1][j] = \min(f[1][j-1] + a_{1,j}, f[2][j-1] + t_{2,j-1} + a_{1,j})$$
$$f[2][j] = \min(f[2][j-1] + a_{2,j}, f[1][j-1] + t_{1,j-1} + a_{2,j})$$

Final Answer:  $\text{exit\_time} = \min(f[1][n] + x_1, f[2][n] + x_2)$  where  $x_i$  is the exit time.

### BOTTOM-UP COMPUTATION

```
f[1][1] = e[1] + a[1][1]
f[2][1] = e[2] + a[2][1]

for j = 2 to n:
    f[1][j] = min(f[1][j-1] + a[1][j], f[2][j-1] + t[2][j-1] + a[1][j])
    f[2][j] = min(f[2][j-1] + a[2][j], f[1][j-1] + t[1][j-1] + a[2][j])
```

**Complexity:** Time:  $O(n)$  | Space:  $O(n)$

## 8. Edit Distance / Levenshtein Distance

---

### PROBLEM EXPLANATION

Given two strings X and Y, compute the minimum number of edit operations (insert, delete, replace) needed to transform X into Y. When the last characters match, we carry over the solution. When they do not, we pick the minimum cost among three operations applied to shorter prefixes.

### CORE FORMULA

$$\begin{aligned}d[i,j] &= j \quad \text{if } i = 0 \\d[i,j] &= i \quad \text{if } j = 0 \\d[i,j] &= d[i-1,j-1] \quad \text{if } X[i] == Y[j] \\d[i,j] &= 1 + \min(d[i-1,j], d[i,j-1], d[i-1,j-1]) \quad \text{if } X[i] \neq Y[j]\end{aligned}$$

### BOTTOM-UP COMPUTATION

```
for i = 0 to m: d[i,0] = i
for j = 0 to n: d[0,j] = j

for i = 1 to m:
  for j = 1 to n:
    if X[i] == Y[j]: d[i,j] = d[i-1,j-1]
    else:           d[i,j] = 1 + min(d[i-1,j], d[i,j-1], d[i-1,j-1])
```

**Complexity:** Time:  $O(m \times n)$  | Space:  $O(m \times n)$

## 9. Longest Palindromic Subsequence

---

### PROBLEM EXPLANATION

Given a sequence X, find the longest subsequence that reads the same forwards and backwards. This problem can be elegantly solved by finding the LCS of X and its reverse version.

### CORE FORMULA

$$LPS(X) = LCS(X, Reverse(X))$$

**Complexity:** Time:  $O(n^2)$  | Space:  $O(n^2)$

## 10. Space-Efficient LCS

### PROBLEM EXPLANATION

Since each row of the LCS table only relies on the previous row's contents, we can optimize the space complexity down to two rows instead of tracking the entire  $m \times n$  matrix.

### SPACE-OPTIMIZED COMPUTATION

```
prev = array[0..n] initialized to 0
curr = array[0..n]

for i = 1 to m:
    curr[0] = 0
    for j = 1 to n:
        if X[i] == Y[j]: curr[j] = prev[j-1] + 1
        else: curr[j] = max(prev[j], curr[j-1])
    prev = curr.clone()
return prev[n]
```

**Complexity:** Time:  $O(m \times n)$  | Space:  $O(\min(m, n))$

## Greedy Algorithm Problems (Chapter 15)

### 1. Activity Selection Problem (CLRS 15.1)

#### PROBLEM EXPLANATION

Select the maximum number of mutually compatible activities from a given set of tasks with explicit start and finish times. The greedy strategy chooses the activity that finishes earliest, leaving maximum room for subsequent activities.

#### FORMAL ALGORITHM

```
ACTIVITY-SELECTION(s, f):
    // Assume activities sorted by finish time:  $f[1] \leq f[2] \leq \dots \leq f[n]$ 
    n = s.length
    A = {1}
    k = 1
    for m = 2 to n:
        if s[m] ≥ f[k]:
            A = A ∪ {m}
            k = m
    return A
```



**Complexity:** Time:  $O(n \log n)$  (sorting) +  $O(n)$  (scan) | Space:  $O(n)$

## 2. Huffman Coding (CLRS 15.3)

---

### PROBLEM EXPLANATION

Constructs an optimal prefix-free binary tree code based on known character frequencies. Huffman's algorithm builds the tree bottom-up by repeatedly combining the two nodes with the smallest frequencies into a new node.

### ALGORITHM

```
HUFFMAN(C):
    n = |C|
    Q = C // Min-heap ordered by frequency
    for i = 1 to n-1:
        z = ALLOCATE-NODE()
        x = EXTRACT-MIN(Q)
        y = EXTRACT-MIN(Q)
        z.freq = x.freq + y.freq
        z.left = x; z.right = y
        INSERT(Q, z)
    return EXTRACT-MIN(Q)
```

**Complexity:** Time:  $O(n \log n)$  | Space:  $O(n)$

## 3. Fractional Knapsack Problem (CLRS 15.2)

---

### PROBLEM EXPLANATION

Unlike the 0/1 Knapsack problem where items cannot be divided, the fractional knapsack allows taking partial amounts. The optimal greedy approach sorts items by their value-to-weight ratio ( $v_i / w_i$ ) and fills the knapsack in descending order.

## ALGORITHM STRATEGY

```
FRACTIONAL-KNAPSACK(w, v, W):
    Sort items in descending order of ratio (v[i] / w[i])
    total_value = 0; remaining = W
    for i = 1 to n:
        if items[i].weight ≤ remaining:
            total_value += items[i].value
            remaining -= items[i].weight
        else:
            total_value += (remaining / items[i].weight) * items[i].value
            break
    return total_value
```

**Complexity:** Time:  $O(n \log n)$  | Space:  $O(n)$

## 4. Job Sequencing with Deadlines

---

### PROBLEM EXPLANATION

Given  $n$  jobs with individual deadlines and profits, choose a sequence to maximize profit. Sort jobs by profit in descending order, and place each job in the latest available time slot before or at its deadline.

**Complexity:** Time:  $O(n^2)$  | Space:  $O(n + \text{ext}\{max\_deadline\})$

## 5. Minimum Spanning Tree - Prim's Algorithm

---

### PROBLEM EXPLANATION

Builds a Minimum Spanning Tree (MST) one vertex at a time. Starting from an arbitrary root node, it greedily selects the minimum-weight edge connecting a vertex inside the current tree to an outside vertex.

### ALGORITHM

```
PRIM(G, w, r):
    for each  $u \in V$ :  $key[u] = \infty$ ,  $\pi[u] = NIL$ ,  $in\_tree[u] = FALSE$ 
     $key[r] = 0$ ;  $Q = \text{Min-Priority-Queue}(V)$ 
    while  $Q$  is not empty:
         $u = \text{EXTRACT-MIN}(Q)$ 
         $in\_tree[u] = TRUE$ 
        for each  $v \in \text{Adj}[u]$ :
            if not  $in\_tree[v]$  and  $w(u, v) < key[v]$ :
                 $\pi[v] = u$ 
                 $key[v] = w(u, v)$ 
                 $\text{DECREASE-KEY}(Q, v, key[v])$ 
```

**Complexity:** Time:  $O(E \log V)$  with Binary Heap | Space:  $O(V)$

## 6. Minimum Spanning Tree - Kruskal's Algorithm

---

### PROBLEM EXPLANATION

Kruskal's algorithm sorts all edges in ascending order of weight and uses a Disjoint-Set structure (Union-Find) to add edges as long as they do not form cycles, uniting independent trees into a single MST.

**Complexity:** Time:  $O(E \log E)$  | Space:  $O(V)$

## 7. Dijkstra's Algorithm

---

### PROBLEM EXPLANATION

Finds single-source shortest paths in a graph with **non-negative edge weights**. It greedily pulls the unvisited vertex with the lowest distance estimate and relaxes its outgoing edges.

```
RELAX(u, v, w):  
    if d[v] > d[u] + w(u, v):  
        d[v] = d[u] + w(u, v)  
        π[v] = u
```

**Complexity:** Time:  $O(E \log V)$  | Space:  $O(V)$

## 8. Task Scheduling to Minimize Weighted Completion Time

---

### PROBLEM EXPLANATION

Minimize  $\sum w_i C_i$  where  $C_i$  is completion time. The optimal greedy choice is to schedule tasks in descending order of the ratio  $w_i / t_i$ .

**Complexity:** Time:  $O(n \log n)$

## 9. Interval Partitioning

---

### PROBLEM EXPLANATION

Process lectures in ascending order of start times. Assign each lecture to an existing compatible classroom with the earliest finish time, or open a new room.

**Complexity:** Time:  $O(n \log n)$

## 10. Elements of Greedy Strategy

---

A greedy algorithm guarantees global optimality if the problem possesses two essential traits:

1. **Greedy Choice Property:** A locally optimal choice can lead to a globally optimal outcome without needing to backtrack.
2. **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its underlying subproblems.

## Comprehensive Problem Comparison

| Problem             | Type   | Core Strategy / Recurrence                              | Time Complexity | Key Property                   |
|---------------------|--------|---------------------------------------------------------|-----------------|--------------------------------|
| Rod Cutting         | DP     | $r(n) = \max(p(i) + r(n-i))$                            | $O(n^2)$        | Overlapping subproblems        |
| Matrix Chain        | DP     | $m[i,j] = \min(m[i,k] + m[k+1,j] + \text{cost})$        | $O(n^3)$        | Chain split structure          |
| LCS                 | DP     | Prefix matching recurrence matrix                       | $O(m \times n)$ | Prefix subproblems             |
| Optimal BST         | DP     | $e[i,j] = \min(e[i,r-1] + e[r+1,j] + w)$                | $O(n^3)$        | Subtree frequency boundaries   |
| LIS                 | DP     | $\text{lis}[i] = 1 + \max(\text{lis}[j])$ for $j < i$   | $O(n^2)$        | Prefix comparisons             |
| Assembly Line       | DP     | $f[i][j] = \min(\text{stay}, \text{switch})$            | $O(n)$          | Sequential stage decisions     |
| Edit Distance       | DP     | $\min(\text{Insert}, \text{Delete}, \text{Substitute})$ | $O(m \times n)$ | String prefix edit chains      |
| Activity Selection  | Greedy | Sort by finish time, pick earliest                      | $O(n \log n)$   | Greedy Choice Property         |
| Huffman Coding      | Greedy | Repeatedly merge two lowest frequencies                 | $O(n \log n)$   | Prefix-free optimality         |
| Fractional Knapsack | Greedy | Sort by value/weight ratio density                      | $O(n \log n)$   | Divisible item packing         |
| Prim's MST          | Greedy | Grow single tree via minimum crossing edge              | $O(E \log V)$   | Cut Property                   |
| Kruskal's MST       | Greedy | Add minimum global edge avoiding cycles                 | $O(E \log E)$   | Cycle Property                 |
| Dijkstra's Paths    | Greedy | Extract minimum unvisited distance estimate             | $O(E \log V)$   | Non-negative weight constraint |

## EXAM REFERENCE REMINDERS

- **DP vs Greedy:** DP evaluates all options at a subproblem stage before choosing. Greedy commits to the local optimum upfront without tracking back or looking at downstream costs.
- **Negative Cycles:** Dijkstra fails in the presence of negative cycles as paths could infinitely minimize costs, violating the greedy extraction finality rule.
- **Space Efficiency:** Always look for row-reduction options when evaluating multi-dimensional DP matrix problems for seen/unseen queries.